Homework-7

1.Write a c program to construct an expression tree for a given expression (a-b)+(c*d).

```
#include<stdio.h>
#include<stdlib.h>
struct node{
char data;
struct node *left,*right;
};
struct node *create(char c){
struct node *n=malloc(sizeof(struct node));
n->data=c;
n->left=n->right=NULL;
return n;
}
void inorder(struct node *root){
if(root){
if(root->left)printf("(");
inorder(root->left);
printf("%c",root->data);
inorder(root->right);
if(root->right)printf(")");
}
}
int main(){
struct node *root=create('+');
root->left=create('-');
```

```
root->left->left=create('a');
root->left->right=create('b');
root->right=create('*');
root->right->left=create('c');
root->right->right=create('d');
inorder(root);
return 0;
}
```

2.Write a c program to check whether the given binary tree is binary search tree.

```
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>
struct node{
int data;
struct node *left,*right;
};
int isBST(struct node *root,long long min,long long max){
if(root==NULL)return 1;
if(root->data<=min||root->data>=max)return 0;
return isBST(root->left,min,root->data)&&isBST(root->right,root->data,max);
}
int main(){
struct node *root=malloc(sizeof(struct node));
root->data=20;
root->left=malloc(sizeof(struct node));
root->right=malloc(sizeof(struct node));
```

```
root->left->data=10;
root->right->data=30;
root->left->left=root->left->right=NULL;
root->right->left=root->right->right=NULL;
if(isBST(root,LLONG_MIN,LLONG_MAX))printf("BST");
else printf("Not a BST");
return 0;
}
```

OUTPUT:

BST

3.Write a c program to implement a max heap.

```
#include<stdio.h>
#define MAX 100
int heap[MAX],n=0;
void insert(int x){
int i;
if(n==MAX)return;
i=n++;
heap[i]=x;
while(i>0&&heap[(i-1)/2]<heap[i]){
int t=heap[i];
heap[i]=heap[(i-1)/2];
heap[(i-1)/2]=t;
i=(i-1)/2;
}
}
```

```
int deleteMax(){
int i=0,largest,left,right,t;
if(n==0)return -1;
int max=heap[0];
heap[0]=heap[--n];
while(1){
largest=i;
left=2*i+1;
right=2*i+2;
if(left<n&&heap[left]>heap[largest])largest=left;
if(right<n&&heap[right]>heap[largest])largest=right;
if(largest==i)break;
t=heap[i];
heap[i]=heap[largest];
heap[largest]=t;
i=largest;
}
return max;
}
void display(){
int i;
for(i=0;i<n;i++)printf("%d ",heap[i]);
printf("\n");
}
int main(){
insert(30);
```

```
insert(10);
insert(50);
insert(20);
insert(40);
display();
printf("Deleted: %d\n",deleteMax());
display();
return 0;
}
```

OUTPUT:

50 40 30 10 20

Deleted: 50

40 20 30 10

4.Given a undirected graph the task is to check ,if there is a cycle in the given graph.

```
#include<stdio.h>
#define MAX 100
int g[MAX][MAX],visited[MAX],n;
int dfs(int v,int parent){
int i;
visited[v]=1;
for(i=0;i<n;i++){
if(g[v][i]){
if(!visited[i]){
if(dfs(i,v))return 1;
}else if(i!=parent)return 1;
}
```

```
}
return 0;
}
int main(){
int m,u,v,i,cycle=0;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++){
scanf("%d%d",&u,&v);
g[u][v]=g[v][u]=1;
}
for(i=0;i<n;i++){
if(!visited[i]&&dfs(i,-1)){
cycle=1;
break;
}
}
if(cycle)printf("Cycle exists");
else printf("No cycle");
return 0;
}
```

OUTPUT:

Cycle exists

5.Given an nXm grid of ‘w’(water) and ‘L’(Land). The task is to count the number of islands.

```
#include<stdio.h>
#define MAX 100
char a[MAX][MAX];
```

```
int n,m;
void dfs(int i,int j){
if(i<0||i>=n||j<0||j>=m||a[i][j]!='L')return;
a[i][j]='w';
dfs(i-1,j);
dfs(i+1,j);
dfs(i,j-1);
dfs(i,j+1);
}
int main(){
int i,j,count=0;
scanf("%d%d",&n,&m);
for(i=0;i<n;i++)
scanf("%s",a[i]);
for(i=0;i<n;i++){
for(j=0;j<m;j++){
if(a[i][j]=='L'){
count++;
dfs(i,j);
}
}
}
printf("%d",count);
return 0;
}
```

OUTPUT:

4

6.Find the minimum spanning tree using kruskals algorithm.

```
#include<stdio.h>
#define MAX 100
struct edge{
int u,v,w;
};
struct edge e[MAX],t;
int parent[MAX];
int find(int x){
while(parent[x]!=x){
parent[x]=parent[parent[x]];
x=parent[x];
}
return x;
}
int main(){
int n,m,i,j,a,b,count=0,cost=0;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++)
scanf("%d%d%d",&e[i].u,&e[i].v,&e[i].w);
for(i=0;i<n;i++)parent[i]=i;
for(i=0;i<m-1;i++){
for(j=0;j<m-i-1;j++){
if(e[j].w>e[j+1].w){
```

```
t=e[j];
e[j]=e[j+1];
e[j+1]=t;
}
}
}
for(i=0;i<m;i++){
a=find(e[i].u);
b=find(e[i].v);
if(a!=b){
parent[a]=b;
printf("%d-%d:%d\n",e[i].u,e[i].v,e[i].w);
cost+=e[i].w;
count++;
}
}
if(count!=n-1)printf("No spanning tree\n");
else printf("Minimum cost=%d\n",cost);
return 0;
}
```

OUTPUT:

```
2-3:4
0-3:5
0-1:10
Minimum cost=19
```

7.Use Dijkstra’s algorithm to find shortest path from a source to all nodes.

```
#include<stdio.h>
#define MAX 100
#define INF 999999
int main(){
int g[MAX][MAX],d[MAX],visited[MAX]={0};
int n,src,i,j,u,min;
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++){
scanf("%d",&g[i][j]);
if(i!=j&&g[i][j]==0)g[i][j]=INF;
}
scanf("%d",&src);
for(i=0;i<n;i++)d[i]=INF;
d[src]=0;
for(i=0;i<n;i++){
u=-1;
min=INF;
for(j=0;j<n;j++){
if(!visited[j]&&d[j]<min){
min=d[j];
u=j;
}
}
if(u==-1)break;
visited[u]=1;
```

```
for(j=0;j<n;j++){
if(!visited[j]&&g[u][j]!=INF&&d[u]+g[u][j]<d[j])
d[j]=d[u]+g[u][j];
}
}
for(i=0;i<n;i++){
if(d[i]==INF)printf("%d:Unreachable\n",i);
else printf("%d:%d\n",i,d[i]);
}
return 0;
}
```

OUTPUT:

0:0

1:4

2:1

3:9

8.Write a program to find a mother vertex in the graph. A mother vertex in a graph G=(V,E) is a vertex V such that all other vertex in G can be reached by a path from V

```
#include<stdio.h>
#define MAX 100
int g[MAX][MAX],visited[MAX],n;
void dfs(int v){
int i;
visited[v]=1;
for(i=0;i<n;i++)
if(g[v][i]&&!visited[i])dfs(i);
```

```
}
int main(){
int m,u,v,i,candidate=-1,valid=1;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++){
scanf("%d%d",&u,&v);
g[u][v]=1;
}
for(i=0;i<n;i++){
if(!visited[i]){
dfs(i);
candidate=i;
}
}
for(i=0;i<n;i++)visited[i]=0;
dfs(candidate);
for(i=0;i<n;i++)
if(!visited[i])valid=0;
if(valid)printf("Mother vertex: %d",candidate);
else printf("No mother vertex");
return 0;
}
```

OUTPUT:

Mother vertex: 0

9.A directed graph is strongly connected if there is a path between all pairs of vertices. N strongly connected components of a directed graph is a maximal strongly of a directed graph .Given a graph print all singly linked list.

```
#include<stdio.h>
#define MAX 100
int g[MAX][MAX],rev[MAX][MAX],visited[MAX],order[MAX],top=0,n;
void dfs1(int v){
int i;
visited[v]=1;
for(i=0;i<n;i++)
if(g[v][i]&&!visited[i])dfs1(i);
order[top++]=v;
}
void dfs2(int v){
int i;
visited[v]=1;
printf("%d ",v);
for(i=0;i<n;i++)
if(rev[v][i]&&!visited[i])dfs2(i);
}
int main(){
int m,u,v,i;
scanf("%d%d",&n,&m);
for(i=0;i<m;i++){
scanf("%d%d",&u,&v);
g[u][v]=1;
rev[v][u]=1;
}
for(i=0;i<n;i++)
```

```
if(!visited[i])dfs1(i);
for(i=0;i<n;i++)visited[i]=0;
while(top>0){
v=order[--top];
if(!visited[v]){
dfs2(v);
printf("\n");
}
}
return 0;
}
```

OUTPUT:

```
0 2 1
3
4
```

10.A bipartite graph G=(V,E) is a graph such that V can lie partitioned into two subsets v1 and v2 and Failed edge has both its vertices in the same subset. Write a graph is a bipartite graph.

```
#include<stdio.h>
#define MAX 100
int g[MAX][MAX],color[MAX],n;
int main(){
int i,j,k,queue[MAX],front,rear;
scanf("%d",&n);
for(i=0;i<n;i++)
for(j=0;j<n;j++)
scanf("%d",&g[i][j]);
```

```
for(i=0;i<n;i++)color[i]=-1;
for(i=0;i<n;i++){
if(color[i]!=-1)continue;
front=rear=0;
queue[rear++]=i;
color[i]=0;
while(front<rear){
k=queue[front++];
for(j=0;j<n;j++){
if(g[k][j]){
if(color[j]==-1){
color[j]=1-color[k];
queue[rear++]=j;
}else if(color[j]==color[k]){
printf("Not Bipartite");
return 0;
}
}
}
}
}
printf("Bipartite");
return 0;
}
```

OUTPUT:

Bipartite